



INFORME DEL TRABAJO PRACTICO

I. PORTADA

Tema:	SW-AyLP-Trabajo-2.1: Persistencia con archivos texto
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero Software “B”
Alumnos participantes:	Sánchez Lema Isaac Adrián
Asignatura:	ALGORITMOS Y LÓGICA DE PROGRAMACIÓN
Docente:	Ing. José Caiza, Mg.

II. INFORME DEL TRABAJO PRACTICO

1.1 Objetivos

General:

Demostrar la importancia de la persistencia de datos en el ciclo de vida del desarrollo de software, mediante la implementación de flujos de lectura (ifstream) y el manejo de excepciones, asegurando la integridad de los recursos y la eficiencia en el procesamiento de archivos externos.

Específicos:

- Explicar la relación entre los flujos de datos (streams) y el almacenamiento físico, detallando las funciones específicas de la clase ifstream para la extracción segura de información desde archivos de texto.
- Describir paso a paso el ciclo de vida de un archivo en C++; desde los métodos de apertura (.open()) y validación de estado (.is open()), hasta el control de flujo mediante estructuras iterativas (while) y la correcta liberación de recursos del sistema mediante el cierre (.close()).
- Desarrollar un algoritmo funcional que automatice la lectura y procesamiento de datos almacenados en el disco duro, permitiendo a la audiencia identificar la utilidad práctica de la persistencia en el desarrollo de software real.

1.2 Modalidad

Presencial.

1.3 Tiempo de duración

Presenciales: 8.

No presenciales: 0.

1.4 Instrucciones

- El trabajo se desarrollará de manera individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y realice la codificación en C++.
- Subir en un documento en formato PDF que contenga capturas del código de cada ejercicio, adicionalmente, comprimir en una carpeta los archivos con el código generado, y subirlo al aula virtual.
- Si se manifiestan dudas se invita a los estudiantes a acudir a las tutorías académicas.

1.5 Listado de equipos, materiales y recursos

- Inteligencia artificial, TAC Inteligencia artificial, TAC Inteligencia artificial, TAC.
- - Computador o laptop.
- - Plataforma educativa de la UTA.



- - Internet para consultas. - Papel - Material de escritorio.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☒ Recursos audiovisuales
- ☐ Gamificación
- ☐ Inteligencia Artificial
- Otros (Especifique): _____

1.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas.

1.7 Resultados obtenidos

Se logró establecer un flujo de comunicación bidireccional entre la memoria volátil y el almacenamiento físico.

- **Logro:** Los datos de los estudiantes (Nombre, Edad, Carrera y Promedio) se almacenan de forma permanente.
- **Evidencia:** Al cerrar y reiniciar el programa, la información permanece disponible, eliminando la pérdida de datos típica de las variables locales.

El uso de la librería <iomanip> permitió generar un reporte estructurado dentro del archivo de texto.

- **Resultado:** El archivo estudiantes.txt mantiene una estructura de columnas alineadas (setw), facilitando su lectura tanto por humanos (en el Bloc de Notas) como por otros sistemas.
- **Validación:** Se implementó un "Encabezado Inteligente" que se autogenera únicamente al crear el archivo, evitando duplicidad de títulos en registros posteriores.

1.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☒ La empatía
- ☒ Pensamiento crítico
- ☐ Flexibilidad
- ☒ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

1.9 Conclusiones

Se concluye que la implementación de la librería <fstream> es el pilar fundamental para transformar programas temporales en aplicaciones funcionales. La capacidad de trasladar datos de la memoria volátil (RAM) al almacenamiento físico permite la creación de sistemas de información reales, donde la integridad de los datos trasciende la ejecución del proceso.



A través del estudio de `ifstream` y el bucle de lectura `while(getline(...))`, se determinó que C++ ofrece herramientas de alta eficiencia para el procesamiento de archivos. Esta metodología no solo optimiza el uso de recursos al leer línea por línea, sino que garantiza que el software sea escalable, permitiendo manejar desde pequeños archivos de texto hasta grandes volúmenes de datos sin comprometer la estabilidad del sistema.

La práctica demostró que una correcta manipulación de archivos requiere obligatoriamente una lógica de validación (uso de `.is_open()`, `try-catch` y `.close()`).

1.10 Recomendaciones

Se recomienda no limitar el programa a buscar archivos en la carpeta raíz del proyecto. En aplicaciones profesionales, es fundamental implementar el manejo de rutas relativas y absolutas, o incluso permitir que el usuario seleccione la ubicación del archivo.

Para proyectos futuros de mayor complejidad, se recomienda evolucionar del uso de archivos de texto plano (.txt) a formatos estandarizados como **CSV** (Comma Separated Values) o **JSON**. Esto facilitaría la interoperabilidad de los datos con otras herramientas como Microsoft Excel, bases de datos SQL o aplicaciones web, permitiendo que la información generada en C++ sea consumida por ecosistemas de software más amplios.

Almacenar datos sensibles en texto plano representa un riesgo de seguridad; por lo tanto, aprender a codificar la información antes de guardarla y decodificarla al leerla es una práctica esencial para cualquier desarrollador de software moderno.

1.11 Referencias bibliográficas

- [1] H. M. Deitel y P. J. Deitel, *C++ Cómo Programar*, 10.^a ed. Ciudad de México, México: Pearson Educación, 2017.
- [2] B. Stroustrup, *The C++ Programming Language*, 4.^a ed. Upper Saddle River, NJ, EE. UU.: Addison-Wesley, 2013.
- [3] Microsoft Learn, "Librería `fstream` (Entrada/Salida de archivos)", Mar. 2024. [En línea]. Disponible en: <https://learn.microsoft.com/es-es/cpp/standard-library/fstream>.
- [4] Universidad Técnica de Ambato, "Guía de Prácticas: Algoritmos y Lógica de Programación", Ambato, Ecuador, 2026.
- [5] I. A. Sánchez Lema, "Deber01_Persistencia_Sanchez_Isaac: Trabajo de Persistencia de Datos en C++", *GitHub*, Abr. 2026. [En línea]. Disponible en: https://github.com/isma1402uta-boop/Deber01_Persistencia_Sanchez_Isaac



1. ANÁLISIS DEL PROBLEMA

Descripción: El sistema actual carece de persistencia; los datos de los estudiantes se pierden al finalizar la ejecución. Se requiere una solución que permita el almacenamiento permanente y la recuperación de información académica.

- **Entradas:** Datos del estudiante (Nombre, Edad, Carrera, Promedio) ingresados por teclado.
- **Procesos:** * Validación de tipos de datos y rangos.

Vínculo con el Sistema de Archivos del SO.

Formateo de cadenas para reporte.

- **Salidas:** Archivo físico estudiantes.txt y reporte estructurado en pantalla.

Diseño de la solución lógica

Estructura: Uso de un struct para agrupar los atributos del objeto Estudiante.

Algoritmo de Escritura:

1. Abrir en modo ios::app.
2. Verificar apertura.
3. Escribir con formato setw.
4. Cerrar flujo.

Algoritmo de Lectura:

1. Abrir con ifstream.
2. Iterar con while(getline).
3. Imprimir cada registro.
4. Cerrar flujo.

Uso de librerías específicas

- **<fstream>** para el flujo de archivos.
- **<iomanip>** para la estética y alineación de los datos.
- **<stdexcept>** para capturar errores de usuario (como meter letras en la edad).

Pruebas de soluciones lógicas dentro del sistema

Prueba de Persistencia: Registrar un estudiante, cerrar el programa, abrirlo y verificar que el dato siga ahí.



Prueba de Resiliencia: Ingresar una edad de -5 o un nombre con números para verificar que las validaciones de las funciones LeerEdad y LeerCadena funcionen.

Prueba de Formato: Abrir el archivo .txt manualmente para confirmar que el encabezado y las columnas están alineadas.

2. DEFINICIÓN DE VARIABLES DENTRO DEL SISTEMA

Tabla 1

Definición de Variables y tipos de Datos del Sistema [1],[2].

CATEGORÍA	NOMBRE DE VARIABLE	TIPO DE DATO	PROPÓSITO / FUNCIÓN
ENTIDAD ESTUDIANTE	nombre	String	Almacena el nombre completo del alumno.
	edad	Int	Guarda la edad del estudiante (validada entre 17-90).
	carrera	String	Almacena el programa académico del usuario.
	promedio	Float	Registra la calificación académica con precisión decimal.
FLUJOS DE ARCHIVO	archivo	Ofstream	Gestiona la salida de datos hacia el disco duro.
	archivoLectura	Ifstream	Gestiona la entrada de datos desde el archivo físico.
CONTROL DE INTERFAZ	opcion	Int	Controla la navegación en el menú del sistema.
	linea	String	Almacena temporalmente cada fila leída del .txt.
VALIDACIÓN Y FLAGS	esNuevo	Bool	Determina si se debe escribir el encabezado del reporte.
	texto	String	Captura temporal de teclado para validar antes de convertir.
	valido	Bool	Flag para confirmar que el formato de entrada es correcto.

"Como se observa en la Tabla 1, la selección de los tipos de datos se basa en los estándares propuestos por Deitel [1] y Stroustrup [2], asegurando el uso eficiente de la memoria y la correcta implementación de la persistencia mediante la librería fstream [3]."



3. ALGORITMO PARA LA GESTIÓN DE ESTUDIANTES MEDIANTE PERSISTENCIA

Fase 1: Inicialización y Control de Interfaz

1. **Inicio del Programa.**
2. **Despliegue del Menú Principal:** Mostrar opciones (1. Registrar, 2. Reporte, 3. Salir).
3. **Captura de Opción:** Leer la decisión del usuario.
 - Si la entrada no es un número, limpiar el buffer y solicitarla nuevamente.
4. **Estructura de Decisión: Evaluar la opción seleccionada.**
 - Si es 1, ir a la Fase 2.
 - Si es 2, ir a la Fase 3.
 - Si es 3, finalizar el programa.

Fase 2: Entrada de Datos y Validación (Registro)

1. **Solicitud de Datos:**
 - **Pedir el Nombre y Carrera.** Aplicar validación para asegurar que solo contengan letras y espacios.
 - **Pedir la Edad.** Validar que sea un número entero y se encuentre en el rango de 17 a 90 años.
 - **Pedir el Promedio.** Validar que sea un valor numérico entre 0.0 y 10.0.
2. **Empaquetamiento:** Almacenar los datos validados dentro de la estructura Estudiante.
3. **Apertura de Flujo de Salida (ofstream):** Intentar abrir el archivo estudiantes.txt en modo anexar (ios::app).
4. **Validación de Persistencia:**
 - Si el archivo es nuevo, escribir el Encabezado del Reporte con formato alineado.
 - Escribir los datos del estudiante usando setw para mantener la simetría de la tabla.
5. **Cierre de Flujo:** Ejecutar. close() para liberar el archivo y asegurar la escritura en disco.

Fase 3: Recuperación y Visualización (Reporte)

1. **Apertura de Flujo de Entrada (ifstream):** Intentar abrir estudiantes.txt en modo lectura.
2. **Control de Existencia:**
 - Si el archivo no existe o no se puede abrir, mostrar mensaje de error: "No



hay registros disponibles".

3. **Lectura Secuencial:** Mientras el lector no llegue al final del archivo (EOF), leer cada línea mediante un bucle while.
4. **Salida por Pantalla:** Imprimir cada línea leída directamente en la consola.
5. **Finalización del Flujo:** Cerrar el archivo .txt.

Fase 4: Cierre del Sistema

1. **Confirmación de Salida:** Mostrar mensaje de despedida.
2. **Liberación de Memoria:** Finalizar la ejecución retornando un valor 0 al Sistema Operativo.
3. **Fin.**
4. **CODIFICACIÓN EN PSEUDOCÓDIGO (PSEINT)**

Algoritmo GestionAcademicaUTA

// Definición de variables globales

Definir opcion Como Entero;

Definir nombre, carrera Como Cadena;

Definir edad Como Entero;

Definir promedio Como Real;

Definir continuar Como Logico;

continuar <- Verdadero;

Mientras continuar Hacer

Limpiar Pantalla;

Escribir "=====";

Escribir " SISTEMA DE GESTION ACADEMICA - UTA ";

Escribir "=====";

Escribir "1. Registrar Nuevo Estudiante";

Escribir "2. Ver Reporte General (Simulado)";



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "3. Salir";

Escribir "-----";

Escribir Sin Saltar "Seleccione una opcion: ";

Leer opcion;

Segun opcion Hacer

1:

Escribir "Ingrese nombre completo:";

Leer nombre;

Repetir

Escribir "Ingrese edad (17 - 90):";

Leer edad;

Si edad < 17 O edad > 90 Entonces

Escribir "Error: Rango no permitido.";

FinSi

Hasta Que edad >= 17 Y edad <= 90;

Escribir "Ingrese carrera:";

Leer carrera;

Repetir

Escribir "Ingrese promedio (0.0 - 10.0):";

Leer promedio;

Si promedio < 0 O promedio > 10 Entonces

Escribir "Error: El promedio debe estar entre 0 y 10.";

FinSi



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Hasta Que promedio ≥ 0 Y promedio ≤ 10 ;

Escribir "";

Escribir "[SISTEMA] Registro validado con exito.";

Escribir "Presione una tecla para continuar...";

Esperar Tecla;

2:

Escribir "";

Escribir

"=====";

Escribir " REPORTE GENERAL DE ESTUDIANTES ";

Escribir

"=====";

Escribir "NOMBRE | EDAD | CARRERA | PROMEDIO";

Escribir "-----";

Escribir nombre, " | ", edad, " | ", carrera, " | ", promedio;

Escribir

"=====";

Escribir "Presione una tecla para volver al menu...";

Esperar Tecla;

3:

Escribir "";

Escribir "Finalizando programa. ¡Exitos en la UTA, Adrian!";

continuar <- Falso;

De Otro Modo:

Escribir "Opcion no valida. Intente de nuevo.";



Esperar 2 Segundos;

Fin Segun

Fin Mientras

FinAlgoritmo

5. DIAGRAMA DE FLUJO

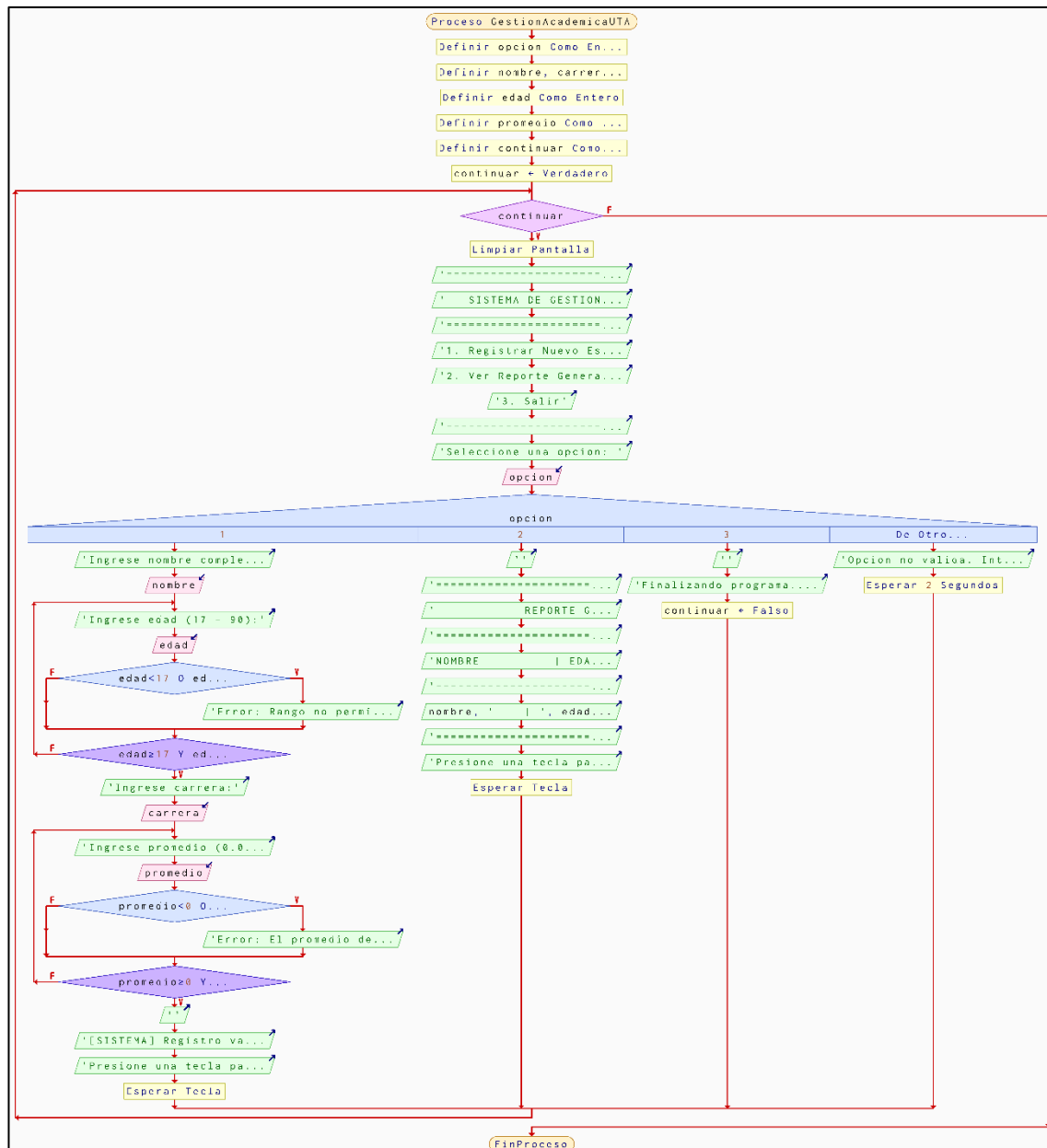


Fig. 1. Diagrama de flujo detallado de la lógica de programación del sistema, generado mediante la herramienta PSeInt. Fuente: Elaboración propia.

6. CODIFICACIÓN EN EL LENGUAJE DE PROGRAMACIÓN C++

```
#include <iostream>
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
#include <string>
```

```
#include <fstream>
```

```
#include <iomanip>
```

```
#include <stdexcept>
```

```
using namespace std;
```

```
// Estructura para registrar la informacion de cada estudiante
```

```
struct estudiantes {
```

```
    string nombre;
```

```
    int edad;
```

```
    string carrera;
```

```
    float promedio;
```

```
};
```

```
// Funcion para validar el nombre del estudiante
```

```
string LeerNombre(string mensaje){
```

```
    string nombreEstudiante;
```

```
    while(true){
```

```
        cout<<mensaje;
```

```
        getline(cin>> ws, nombreEstudiante);
```

```
        bool valido = true;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
for(char n : nombreEstudiante){  
    if(!isalpha(n) && !isspace(n)){  
        valido = false;  
        break;  
    }  
}  
  
if(nombreEstudiante.empty()){  
    cout<<"Error: El nombre del estudiante se encuentra vacio."<<endl;  
    continue;  
}else if(!valido){  
    cout<<"Error: El nombre del estudiante contiene caracteres invalidos."<<endl;  
}else{  
    return nombreEstudiante;  
}  
}  
}
```

// Funcion para validad la edad del estudiante

```
int LeerEdad(string mensaje, int min, int max){  
    int edad;  
    string texto;  
  
    while(true){  
        cout<<mensaje;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
getline(cin>> ws, texto);
```

```
if(texto.empty()){
```

```
    cout<<"Error: El campo se encuentra vacio, Ingrese una edad."<<endl;
```

```
    continue;
```

```
}
```

```
try{
```

```
    edad = stoi(texto);
```

```
    if(edad > max || edad < min){
```

```
        cout<<"Error: Ingrese una edad dentro del rango de "<<min<<" a "<<max<<"."<<endl;
```

```
        continue;
```

```
    }
```

```
    return edad;
```

```
}catch(invalid_argument& e){
```

```
    cout<<"Error: Ingrese una edad valida."<<endl;
```

```
}catch(out_of_range& e){
```

```
    cout<<"Error: La edad es demasiado grande."<<endl;
```

```
}catch(...){
```

```
    cout<<"Error: Ocurrio un error al procesar la edad."<<endl;
```

```
}
```

```
}
```

```
}
```

```
// Funcion para validar el promedio del estudiante
```



```
float LeerPromedio(string mensaje, float min, float max){

    float promedio;

    string texto;

    while(true){

        cout<<mensaje;

        getline(cin>> ws, texto);

        if(texto.empty()){

            cout<<"Error: El campo se encuentra vacio, Ingrese un promedio."<<endl;

            continue;

        }

        try{

            promedio = stof(texto);

            if(promedio > max || promedio < min){

                cout<<"Error: Ingrese un promedio dentro del rango de "<<min<<" a "<<max<<"
                ."<<endl;

                continue;

            }

            return promedio;

        }catch(invalid_argument& e){

            cout<<"Error: Ingrese un promedio valido."<<endl;

        }catch(out_of_range& e){

            cout<<"Error: El promedio es demasiado grande."<<endl;

        }catch(...){

            cout<<"Error: Ocurrio un error al procesar el promedio."<<endl;

        }

    }

}
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



}

}

// Funcion para validar la carrera del estudiante

```
string LeerCarrera(string mensaje){
```

```
    string carrera;
```

```
    while(true){
```

```
        cout<<"mensaje";
```

```
        getline(cin>> ws, carrera);
```

```
        bool valido = true;
```

```
        for(char n : carrera){
```

```
            if(!isalpha(n) && !isspace(n)){
```

```
                valido = false;
```

```
                break;
```

```
            }
```

```
        }
```

```
        if(carrera.empty()){
```

```
            cout<<"Error: La carrera del estudiante se encuentra vacia."<<endl;
```

```
            continue;
```

```
        }else if(!valido){
```

```
            cout<<"Error: La carrera del estudiante contiene caracteres invalidos."<<endl;
```

```
        }else{
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
        return carrera;
    }
}
}

// Almacenar la informacion del estudiante dentro del archivo de texto

void GuardarRegistro(const string& nombre, int edad, const string& carrera, float promedio) {

    ifstream prueba("estudiantes.txt");

    bool archivoNuevo = !prueba.is_open();

    prueba.close();

    ofstream archivo;

    archivo.open("estudiantes.txt", ios::app);

    // Verificación de apertura del archivo.
    if (!archivo.is_open()) {
        throw runtime_error("Error critico: No se pudo acceder al archivo de persistencia.");
    }

    if (archivoNuevo) {

        archivo
        "=====
===== " << endl;

        archivo << "          REPORTE DE ESTUDIANTES REGISTRADOS          " <<
        endl;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
archivo << endl;

"=====
====" << endl;

archivo << left << setw(20) << "NOMBRE" << " | "

    << setw(4) << "EDAD" << " | "

    << setw(20) << "CARRERA" << " | "

    << "PROMEDIO" << endl;

archivo << "-----" << endl;

}

archivo << left << setw(20) << nombre << " | "

    << setw(4) << edad << " | "

    << setw(20) << carrera << " | "

    << fixed << setprecision(2) << promedio << endl;

// Cierre obligatorio del flujo de datos.

archivo.close();

cout << "\n[SISTEMA] Persistencia completada con exito." << endl;

}

// Metodo para mostrar el reporte de los estudiantes registrados

void MostrarReporte() {

    ifstream archivo("estudiantes.txt");

    string linea;
```



```
if (!archivo.is_open()) {  
    cout << "\n[!] No hay registros guardados todavia." << endl;  
    return;  
}  
  
cout << endl;  
  
while (getline(archivo, linea)) {  
    cout << linea << endl;  
}  
  
archivo.close();  
  
cout  
===== <<  
=====\n" << endl;  
}  
  
  
// Metodo para mostrar el reporte de los estudiantes registrados  
  
void EjecutarRegistro() {  
  
    // Variables locales para mayor orden  
  
    string nombre = LeerNombre("Ingrese el nombre completo: ");  
    int edad = LeerEdad("Ingrese la edad (17-90): ", 17, 90);  
    string carrera = LeerCarrera("Ingrese la carrera: ");  
    float promedio = LeerPromedio("Ingrese el promedio (0.0 - 10.0): ", 0, 10);
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
try {

    GuardarRegistro(nombre, edad, carrera, promedio);

} catch (const exception& e) {

    cout << "\a" << e.what() << endl; // \a emite un pitido de alerta

}

}

// Metodo para mostrar el menu principal del sistema

void MostrarMenu() {

    cout << "\n===== " << endl;

    cout << "  SISTEMA DE GESTION ACADEMICA - UTA  " << endl;

    cout << "===== " << endl;

    cout << "1. Registrar Nuevo Estudiante" << endl;

    cout << "2. Ver Reporte General" << endl;

    cout << "3. Salir" << endl;

    cout << "-----" << endl;

    cout << "Seleccione una opcion: ";

}

// Metodo principal del programa

int main() {

    int opcion;
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
do {

    MostrarMenu();

    if (!(cin >> opcion)) {

        cout << "\n[!] Error: Entrada no valida. Ingrese un numero." << endl;

        cin.clear();

        cin.ignore(1000, '\n');

        continue;

    }

    // Limpiar el salto de linea que deja cin >> antes de ir a los getline

    cin.ignore(1000, '\n');

    switch (opcion) {

        case 1:

            EjecutarRegistro();

            break;

        case 2:

            MostrarReporte();

            break;

        case 3:

            cout << "\n[INFO] Finalizando programa. Datos asegurados." << endl;

            break;

        default:

            cout << "\n[!] Error: Opcion fuera del rango (1-3)." << endl;

    }

} while (opcion != 3);
```



```
return 0;
```

```
}
```

7. PRUEBAS DE SOLUCIÓN DEL SISTEMA DESARROLLADO

CASO 1

```
=====
SISTEMA DE GESTION ACADEMICA - UTA
=====
1. Registrar Nuevo Estudiante
2. Ver Reporte General
3. Salir
-----
Seleccione una opcion: 1
Ingrese el nombre completo: SANCHEZ LEMA ISAAC ADRIAN
Ingrese la edad (17-90): 18 AÑOS
Ingrese la carrera: SOFTWARE
Ingrese el promedio (0.0 - 10.0): 9.00

[SISTEMA] Persistencia completada con exito.

=====
SISTEMA DE GESTION ACADEMICA - UTA
=====
1. Registrar Nuevo Estudiante
2. Ver Reporte General
3. Salir
-----
Seleccione una opcion: 2

=====
REPORTE DE ESTUDIANTES REGISTRADOS
=====
NOMBRE | EDAD | CARRERA | PROMEDIO
-----
SANCHEZ LEMA ISAAC ADRIAN | 18 | SOFTWARE | 9.00
=====
```

Fig. 2. Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado [1]. Fuente: Elaboración propia.



CASO 2

```
=====
  SISTEMA DE GESTION ACADEMICA - UTA
=====
1. Registrar Nuevo Estudiante
2. Ver Reporte General
3. Salir
-----
Seleccione una opcion: 1
Ingrese el nombre completo: i3432
Error: El nombre del estudiante contiene caracteres invalidos.
Ingrese el nombre completo:
q_234edxcvbnmj,mjhgftre
Error: El nombre del estudiante contiene caracteres invalidos.
Ingrese el nombre completo:
GARCIA LEMA MELANY SARAHI
Ingrese la edad (17-90): 18
Ingrese la carrera: ODONTOLOGIA
Ingrese el promedio (0.0 - 10.0): 9.43

[SISTEMA] Persistencia completada con exito.

=====
  SISTEMA DE GESTION ACADEMICA - UTA
=====
1. Registrar Nuevo Estudiante
2. Ver Reporte General
3. Salir
-----
Seleccione una opcion: 2

=====
                        REPORTE DE ESTUDIANTES REGISTRADOS
=====
NOMBRE                | EDAD | CARRERA                | PROMEDIO
-----
SANCHEZ LEMA ISAAC ADRIAN | 18   | SOFTWARE                | 9.00
GARCIA LEMA MELANY SARAHI | 18   | ODONTOLOGIA             | 9.43
=====
```

Fig. 3. Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado mediante validaciones de entradas [1]. Fuente: Elaboración propia.



CASO 3

```
=====
  SISTEMA DE GESTION ACADEMICA - UTA
=====
1. Registrar Nuevo Estudiante
2. Ver Reporte General
3. Salir
-----
Seleccione una opcion: 1
Ingrese el nombre completo: SEGOVIA GARCIA JOSEPH ANDE
Ingrese la edad (17-90): 20
Ingrese la carrera: SOFTWARE
Ingrese el promedio (0.0 - 10.0):
1

[SISTEMA] Persistencia completada con exito.

=====
  SISTEMA DE GESTION ACADEMICA - UTA
=====
1. Registrar Nuevo Estudiante
2. Ver Reporte General
3. Salir
-----
Seleccione una opcion: 2

=====
                        REPORTE DE ESTUDIANTES REGISTRADOS
=====
NOMBRE                      | EDAD | CARRERA                      | PROMEDIO
-----
SANCHEZ LEMA ISAAC ADRIAN | 18   | SOFTWARE                     | 9.00
GARCIA LEMA MELANY SARAHI | 18   | ODONTOLOGIA                  | 9.43
TOAQUIZA ANCHAPAXI SANTIAGO ALEXANDER | 19   | OJMKOEIJD                    | 4.00
SEGOVIA GARCIA JOSEPH ANDE | 20   | SOFTWARE                     | 1.00
=====
```

Fig. 4. Interfaz de usuario y ejecución del sistema de gestión académica en entorno de desarrollo integrado mediante validación de datos de entrada múltiple [1]. Fuente: Elaboración propia.

REPORTE FINAL DENTRO DEL ARCHIVO DE TEXTO

REPORTE DE ESTUDIANTES REGISTRADOS			
NOMBRE	EDAD	CARRERA	PROMEDIO
SANCHEZ LEMA ISAAC ADRIAN	18	SOFTWARE	9.00
GARCIA LEMA MELANY SARAHI	18	ODONTOLOGIA	9.43
TOAQUIZA ANCHAPAXI SANTIAGO ALEXANDER	19	OJMKOEIJD	4.00
SEGOVIA GARCIA JOSEPH ANDE	20	SOFTWARE	1.00

Fig. 5. Reporte estructurado de estudiantes registrados con persistencia de datos en formato de texto plano [3]. Fuente: Elaboración propia.